

Computational Economics

Lab: Introduction to Git

Version control, project history, and collaboration

Yaolang Zhong yaolang.zhong@e.u-tokyo.ac.jp
Graduate School of Economics, The University of Tokyo

Three Cases

- ▶ **Case: Keep earlier versions**

You keep revising a file but worry that saving your latest edits will erase useful earlier work. Soon you have v1, v2, v3, ... What changed, and which version should you use?

With Git: record versions in one project and revisit earlier work.

- ▶ **Case: Work with others**

You and a partner edit the same project. You exchange files and explain each change in messages or meetings, then try to combine your edits without losing either person's work.

With Git: compare changes, see who made them, and merge contributions.

- ▶ **Case: Build on an existing project**

You find a useful project online and want to try it or adapt it. After downloading a copy, its author releases improvements. How do you bring those updates into your own copy?

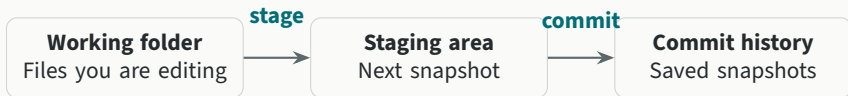
With Git: clone the repository and fetch later updates to integrate.

Git: Keep Earlier Versions

Git is software you install on your computer to keep a history of your work. It helps you record versions, revisit earlier work, and combine changes.

- ▶ Start with an ordinary working folder containing your files. **Initialize** Git with `git init`: the folder becomes a **repository**, a project whose history Git can record.
- ▶ A **commit** is like a snapshot of the project's recorded files at a particular point. You can return to that version or recover useful content from it.
- ▶ Each commit has a **hash**: an identifier made of letters and numbers, often displayed in a short form such as `a3f7c2d`. You also write a message, such as “Add a plotting function.”
- ▶ Git keeps these snapshots in its history, so you can keep working in one folder instead of making `v1`, `v2`, `v3`, ...

Git: Record a Version



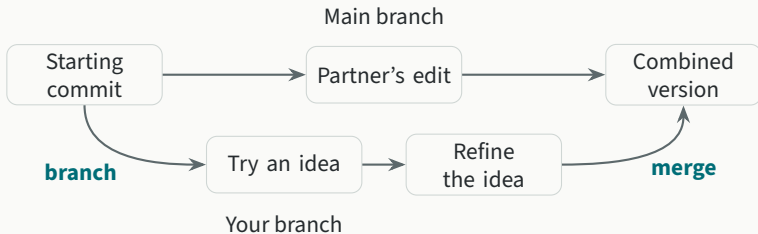
- ▶ **Edit and save:** your working files change, but no new snapshot is recorded in Git yet.
- ▶ **Stage:** choose which edits go into the next snapshot. Other tracked files retain their previously recorded content in that snapshot.
- ▶ **Commit:** record the prepared snapshot in history, with your message. Later edits leave that snapshot unchanged.

Case: Two files changed, only one is ready

You edit `code.py` and `notes.txt`, but stage only `code.py`. The commit records the code changes; your unfinished edits to `notes.txt` stay in the working folder for later.

Git: Branch and Merge

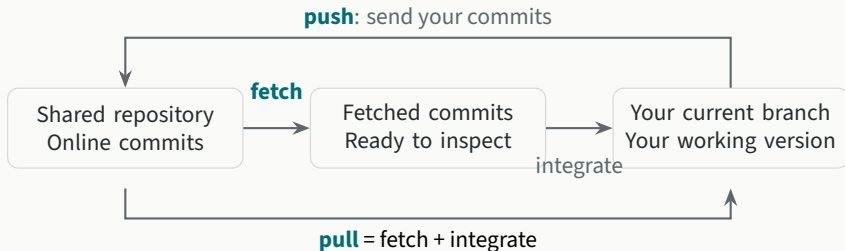
A **branch** lets you try an idea separately while a partner works on the main version.



- ▶ Each box is a **commit**. Both branches start from the same history, then record separate changes.
- ▶ **Merge**: bring the changes from your branch into the main branch.
- ▶ A **conflict** means Git cannot combine edits automatically. Decide which content to keep, then complete the merge.

Git: Push, Fetch, and Pull

Exchange commits with your partner through a shared repository.



- ▶ **Push:** send your commits to the shared repository.
- ▶ **Fetch:** download new commits to inspect. Your current branch and working files stay unchanged.
- ▶ **Pull:** fetch and integrate updates into your current branch, using a merge or replaying your commits on top, depending on your settings. Resolve conflicts if needed.

GitHub: Start from an Existing Project

GitHub is an online platform for storing and sharing Git repositories. It gives a project a web address where you can browse its files and history.

- ▶ Your **local repository** is on your computer. A repository hosted on GitHub can serve as the shared copy for a project.
- ▶ A **remote** is a saved name and address for another repository. Git uses this connection when you push, fetch, or pull.
- ▶ Read the project's **README**: the introductory file explaining what the project does and how to use it.
- ▶ Run **git clone** with the repository's address to create a local folder with its files and history. Git also sets up a remote connection to the original repository.
- ▶ Work in your copy, make commits, and fetch later updates. Pushing to the original repository requires permission from its owner.

One Small Change, One Clear Commit

Case: Improve the README

1. Open the project folder in VS Code and inspect its Source Control panel.
2. Add a sentence explaining how to run the program.
3. Inspect the **diff**: the comparison between old and new content.
4. Follow the new instructions to check that they work.
5. Stage the intended files and commit with a useful message, such as “Add instructions for running the program.”

Before committing, check that the diff contains only the changes you intended.

Choose What Belongs in the Repository

Track and share

Source code: `.py` and `.ipynb`
A README explaining how to run the work
A list of required packages
Small, relevant results when requested

Keep outside the shared history

Local environment: `.venv/`
Caches and notebook checkpoints
Passwords, access tokens, and private data
Large generated files that can be recreated

- ▶ A **.gitignore** file lists patterns for files Git should leave untracked.
- ▶ Ignoring an already tracked file does not remove it from the repository or its history.
- ▶ Check notebook outputs as well as code before pushing. A public repository makes its tracked content accessible to others.

Connect Your Folder to GitHub

1. Install Git, register for GitHub, and open your project folder in VS Code.
2. In Source Control, initialize the folder as a Git repository. Set your commit name and email when prompted.
3. Add a README and a `.gitignore`; inspect, stage, and commit them.
4. Choose **Publish to GitHub**, sign in through the browser, choose a repository name and visibility, and check which files are included.
5. Open the resulting GitHub URL and confirm that your commit and files appear.

This workflow creates the GitHub repository and connects the local folder. If you already have a GitHub repository, clone it first and work in that local folder.

Practice: Is the Change on GitHub Yet?

Case: A Missing README Update

You edit and save the README, make a local commit, and open GitHub. The website still shows the old README.

- ▶ Which steps have completed: save, stage, commit, push?
- ▶ What would you check before trying again?
- ▶ After pushing, where can you verify the commit message and changed file?

Repeat with a second small edit. Explain the difference between the local history and the remote history to a partner.

References and Further Reading

1. **Prof. Feng Zhigang (2026), AI for Economic Research: Dynamic Models, Language, and Agents. [Online course]**
 - ▶ Session 1.5: The Workbench: Python, VS Code, Git and GitHub
2. **Microsoft, Visual Studio Code Documentation. [Webpages]**
 - ▶ Quickstart: use source control in VS Code
3. **GitHub, GitHub Docs. [Webpage]**
 - ▶ Creating an account on GitHub
4. **Scott Chacon and Ben Straub, Pro Git. 2nd ed. [Book]**
 - ▶ Installing Git